



**SOMAIYA**

VIDYAVIHAR UNIVERSITY

**K J Somaiya School of Engineering**

(formerly K J Somaiya College of Engineering)



# Automated Failure Detection & Alert Notification for Containerized Applications

DevOps IA-2 Case Study, Tool: Prometheus and Alertmanager

Yashasvi Gupta · 16010123341

Shweta Karandikar · 16010123329

Aditi Agrawal · 16010123018



# Containers fail quietly



## A process hangs

docker ps still says Up, but users time out



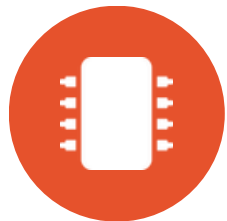
## A dependency dies

the web service keeps answering, with errors



## Memory leaks

until the kernel's OOM killer stops the process



## CPU hits its limit

the container is throttled and everything slows



## It crashes and restarts

the restart hides the problem

## Our goal

1. Detect each of these automatically
2. Notify the right person within about a minute
3. One message per incident, not a flood
4. Say when it is fixed
5. Measure how well the tool does it

*Our setting: a hospital's ordering system. If it fails at 2 am and nobody is told, a ward waits for insulin that never comes.*

# Why Prometheus and Alertmanager



## Built for containers

CNCF graduated; discovers Docker and Kubernetes targets



## PromQL

rates, ratios and percentiles in one query language



## Noise control

grouping, routing, inhibition and silences



## Configuration as code

YAML in Git, unit-tested with promtool and amtool



## Free and open source

Apache 2.0, self-hosted, no per-host pricing



## Light

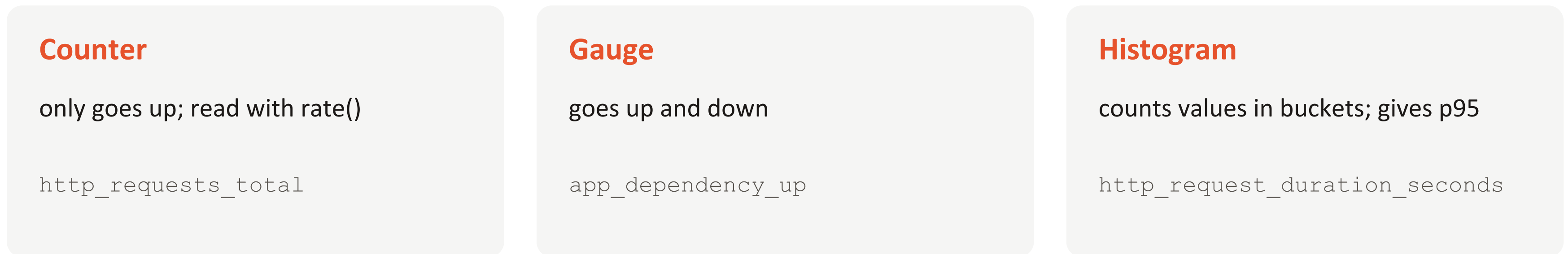
the whole monitoring side used about 180 MiB

*Also considered: Nagios, Zabbix, Datadog, Grafana Alerting, AWS CloudWatch (compared later)*

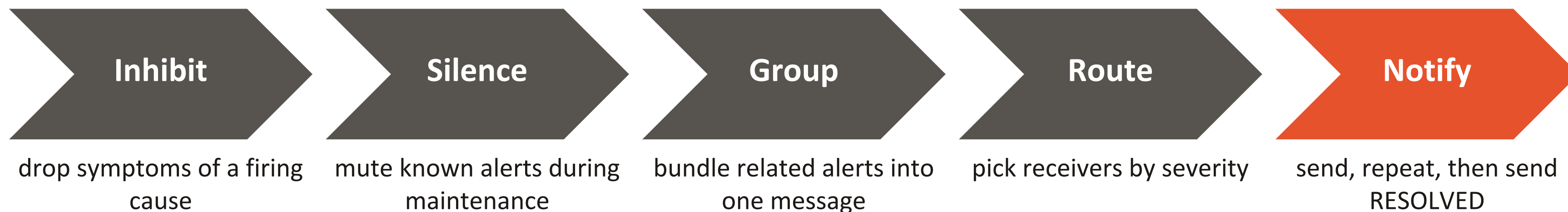
# How Prometheus works



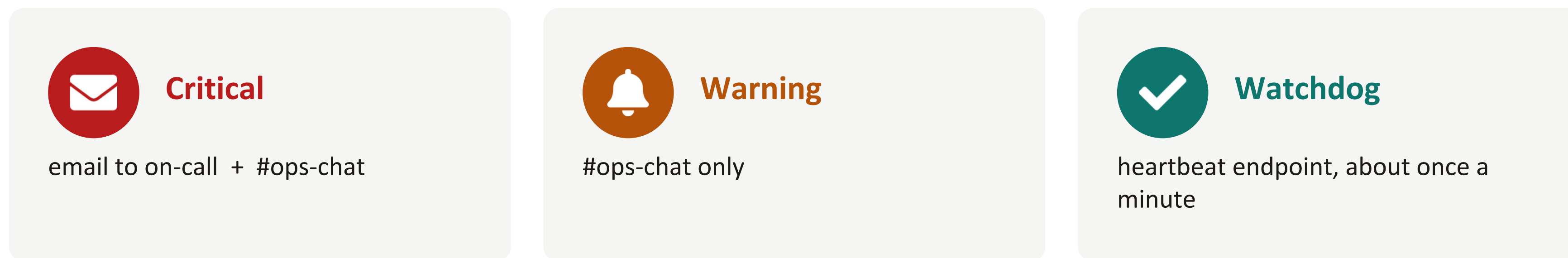
## Metric types we use



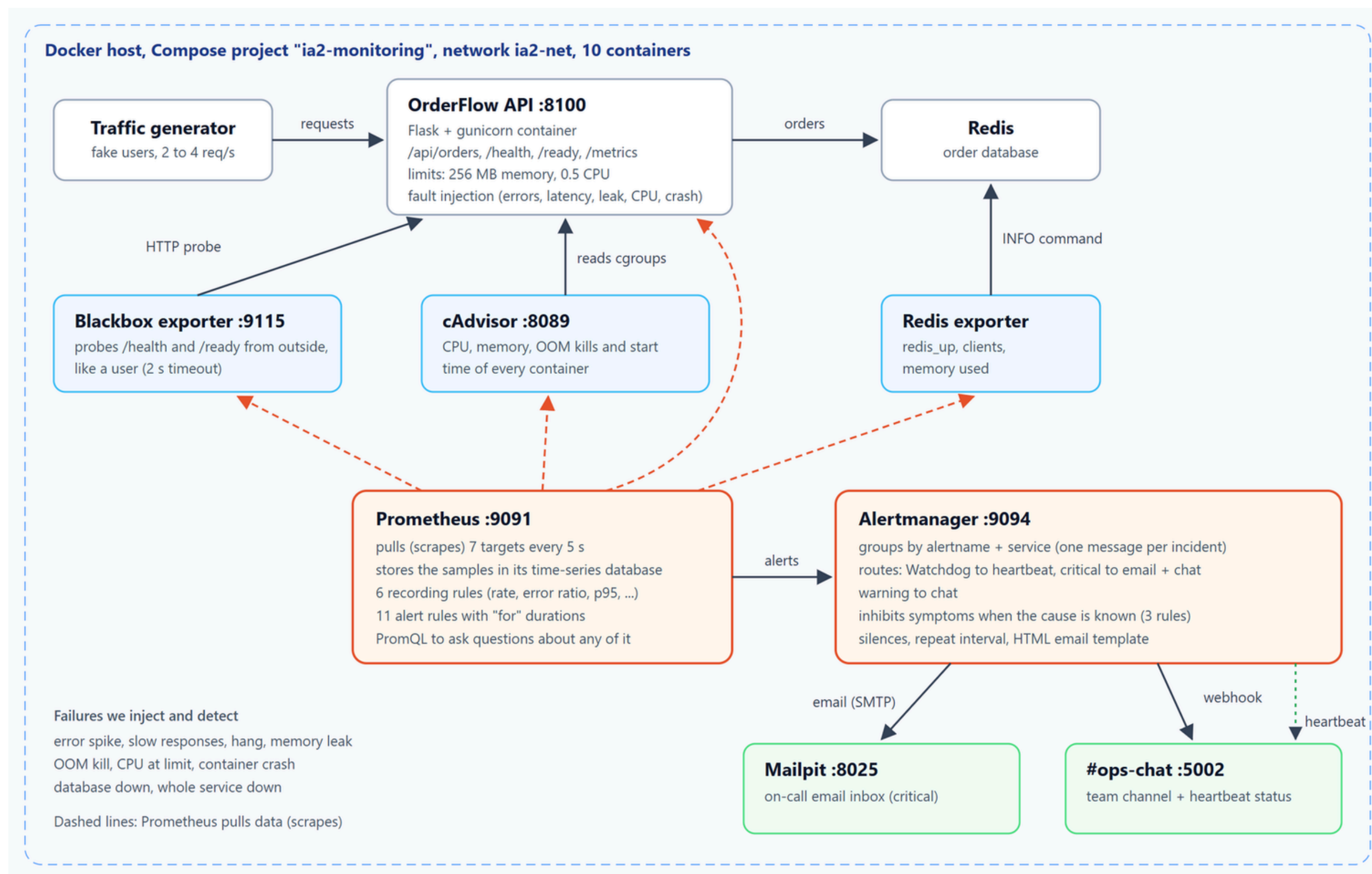
# How Alertmanager handles alerts



## Our routes



# Our setup: ten containers, one command



10

containers via docker compose

7

scrape targets, every 5 s

6 + 11

recording + alert rules

3

receivers: email, chat, heartbeat

# Three angles of monitoring



## White-box

the app reports on itself through  
/metrics

### Sees

request counts, latency histogram,  
Redis status

### Catches

error spikes and slowness



## Black-box

the blackbox exporter calls /health  
and /ready like a user

### Sees

probe\_success, probe duration

### Catches

a hang: /metrics still answers (up = 1)  
but users time out



## Container-level

cAdvisor reads every container's  
cgroups

### Sees

memory vs limit, CPU throttling, OOM  
kills, restarts

### Catches

leaks, OOM kills, CPU starvation,  
crashes

# Eleven alert rules

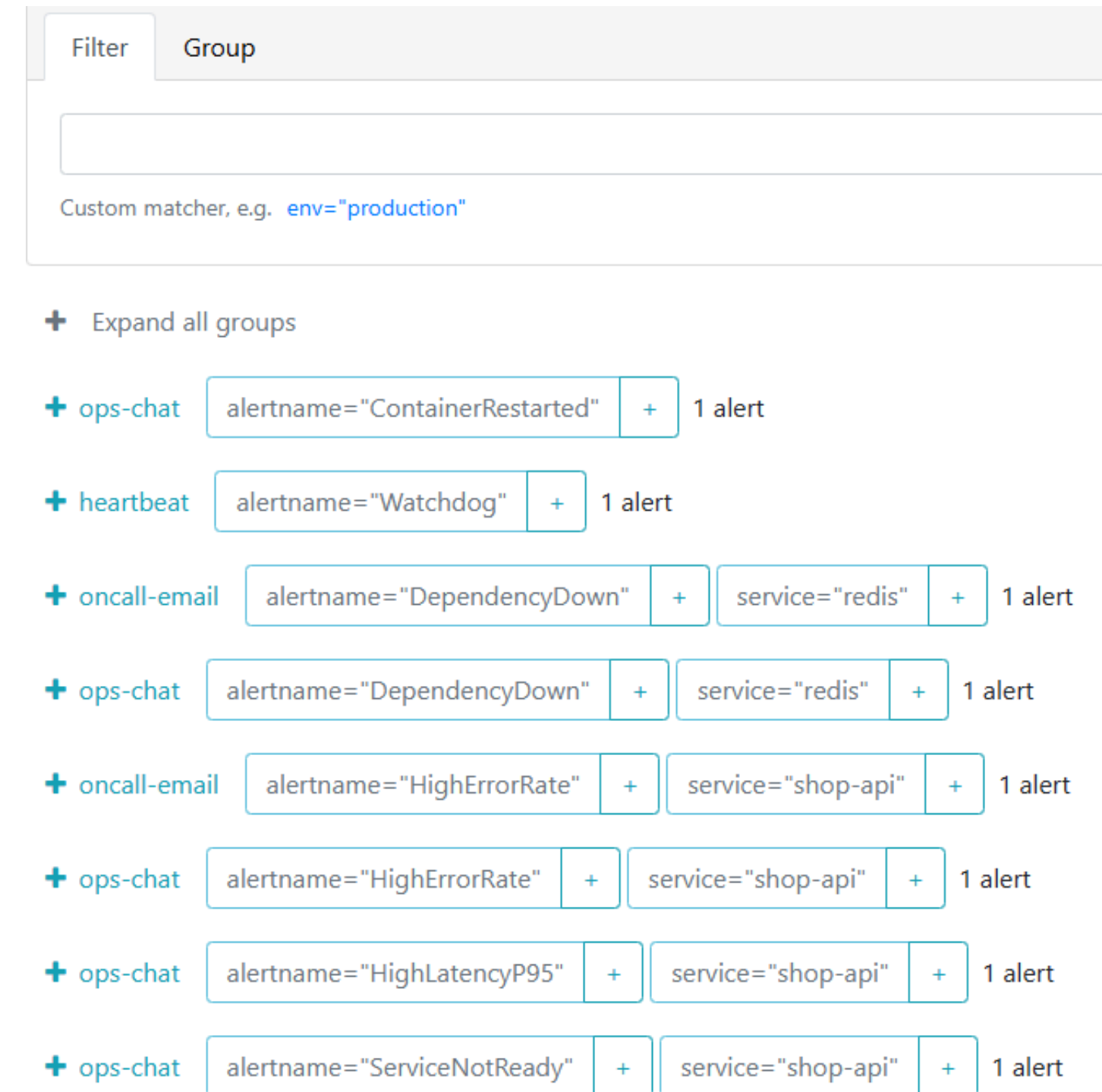
| Alert                    | Fires when                                            | Severity |
|--------------------------|-------------------------------------------------------|----------|
| ServiceDown              | up == 0 for 30 s                                      | critical |
| EndpointDown             | black-box probe of /health fails for 40 s             | critical |
| DependencyDown           | Redis down, or its exporter unreachable, for 15 s     | critical |
| HighErrorRate            | more than 10% of requests return 5xx, for 30 s        | critical |
| ContainerOOMKilled       | the kernel killed a process for using too much memory | critical |
| ServiceNotReady          | /ready probe fails for 45 s                           | warning  |
| HighLatencyP95           | p95 latency above 500 ms for 1 min                    | warning  |
| ContainerMemoryNearLimit | memory above 80% of the container limit for 15 s      | warning  |
| ContainerCPUThrottled    | throttled in more than half of CPU periods for 30 s   | warning  |
| ContainerRestarted       | a container started again in the last 5 min           | warning  |
| Watchdog                 | always firing: proves the alert path works            | none     |

# One message per incident

## Order database (Redis) stops:



- 4 alerts raised in Prometheus (3 firing, 1 pending); on-call got 1 email about the cause.
- Inhibition: a firing cause hides its symptoms.
- Grouping: related alerts share one message.
- Causes wait less than symptoms (15 s vs 30 s), so the cause is already firing when symptoms arrive.



Filter Group

Custom matcher, e.g. `env="production"`

+ Expand all groups

- + ops-chat alertname="ContainerRestarted" + 1 alert
- + heartbeat alertname="Watchdog" + 1 alert
- + oncall-email alertname="DependencyDown" + service="redis" + 1 alert
- + ops-chat alertname="DependencyDown" + service="redis" + 1 alert
- + oncall-email alertname="HighErrorRate" + service="shop-api" + 1 alert
- + ops-chat alertname="HighErrorRate" + service="shop-api" + 1 alert
- + ops-chat alertname="HighLatencyP95" + service="shop-api" + 1 alert
- + ops-chat alertname="ServiceNotReady" + service="shop-api" + 1 alert

*Alertmanager during the outage, with Inhibited ticked: the symptoms are held back*

# What on-call and the team see

From <alertmanager@orderflow.local>  
To <oncall@orderflow.local>  
Subject **[FIRING x1] DependencyDown on redis**  
Date Sat, 3 Oct 2026, 1:22 am (2.5 kB)  
Tags Add tags...

HTML HTML Source Text Headers Raw HTML Check 91% Link Chec

## Something needs attention: DependencyDown

OrderFlow monitoring · 1 firing, 0 resolved

|            |                                                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------------------------------|
| Status     | FIRING                                                                                                                             |
| Summary    | Redis (order database) is down                                                                                                     |
| Details    | The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.                                                      |
| What to do | Check the ia2-redis container: `docker ps -a`, `docker start ia2-redis`.                                                           |
| Started    | 2026-10-02 19:52:12 UTC                                                                                                            |
| Labels     | alertname=DependencyDown app=orderflow environment=ia2-demo instance=redis-exporter:9121 job=redis service=redis severity=critical |

### On-call email

Subject and HTML body come from our template, including a "what to do" line.

### # ops-chat

20 firing · 18 resolved notifications in this channel · page refreshes every 5 s

**AM** Alertmanager 19:52:26 UTC · #ops-chat

**FIRING DependencyDown** · critical · redis  
Redis (order database) is down  
The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.

**AM** Alertmanager 19:50:36 UTC · #ops-chat

**FIRING ContainerRestarted** · warning · ia2-redis  
Container ia2-redis restarted  
ia2-redis started again in the last 5 minutes (crash, OOM kill or manual restart).

**RESOLVED ContainerRestarted** · warning · ia2-shop-api  
Container ia2-shop-api restarted  
ia2-shop-api started again in the last 5 minutes (crash, OOM kill or manual restart).

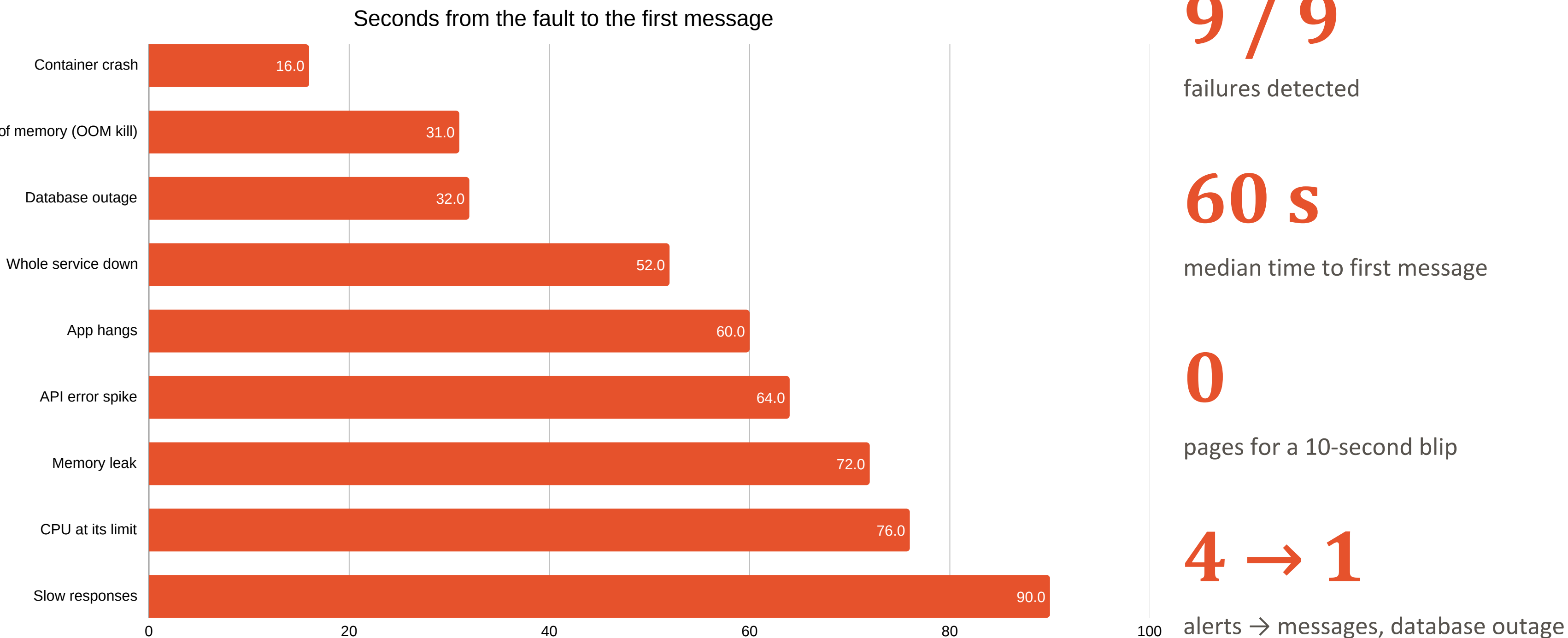
**AM** Alertmanager 19:50:06 UTC · #ops-chat

**RESOLVED DependencyDown** · critical · redis  
Redis (order database) is down  
The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.

### #ops-chat

Every warning and critical alert appears here, first as FIRING, then as RESOLVED.

# Every failure detected within 90 seconds



# How it compares

|                     | Prometheus + Alertmanager      | Nagios Core        | Zabbix               | Datadog            | CloudWatch               |
|---------------------|--------------------------------|--------------------|----------------------|--------------------|--------------------------|
| Cost                | free, open source              | free; XI is paid   | free, open source    | paid per host      | pay per metric and alarm |
| How data arrives    | pull over HTTP                 | plugin checks      | agents, SNMP         | agent to SaaS      | AWS services, agent      |
| Fit for containers  | native discovery, cAdvisor     | weak: static hosts | Docker template      | strong             | ECS and EKS              |
| Alert noise control | group, inhibit, silence, route | escalations        | actions, escalations | notification rules | alarms to SNS            |
| Config as code      | YAML + unit tests              | text files         | mostly web UI        | UI, Terraform      | CloudFormation           |
| Logs and traces     | no (add Loki, Tempo)           | no                 | limited              | yes                | yes                      |

Summarised from each product's public documentation, October 2026. Grafana Alerting builds on data sources such as Prometheus.

# Strengths, limits and lessons



## Strengths

- One query language for graphs and alerts
- Alerts are code: reviewed, tested in CI
- Grouping and inhibition cut the noise
- Small footprint (about 180 MiB)



## Limitations

- Metrics only: no logs or traces
- One server is a single point of failure
- Local storage, not long-term
- PromQL takes time to learn



## Bugs our tests found

- CPU alert never fired: the usage ratio was too jumpy, so we alert on throttling instead
- Database alert flapped: the exporter timed out, so we added a 1 s timeout and alert on up == 0 too

# Tested like code: the CI pipeline



## git push

GitHub Actions starts on every push to main



## Unit tests

20 app tests, 6 receiver tests, flake8



## Rule tests

8 promtool cases, amtool routing checks



## Start the stack

all 10 containers, wait for targets and heartbeat



## Break it

inject 50% errors; pass only if chat AND email get the alert

**Why this matters:** a broken alert rule is as bad as broken code, because nobody hears about the next outage. Every change to a rule, a route or a template is tested before it can reach main, including one test proving a 10-second blip does not page anyone.

# Conclusion



## Detects

all 9 failure types, from a crash to a slow memory leak, in 16 to 90 seconds



## Filters

one message per incident; a 10-second blip pages nobody



## Closes the loop

resolved messages, a heartbeat, and tests in CI for every rule

**Github:** [https://github.com/Yashvi2874/DevOps\\_IA2](https://github.com/Yashvi2874/DevOps_IA2)

**Drive:** [https://drive.google.com/drive/folders/11pqw2HeYxWI5QLId-3zkHv1x-a\\_rm6BP?usp=sharing](https://drive.google.com/drive/folders/11pqw2HeYxWI5QLId-3zkHv1x-a_rm6BP?usp=sharing)

# Thank you